Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 3: Data types, text files and data import

Dr. Aurélien Sallin (Lecturer) Ana Armendariz (Tutor)

15 October 2026

Table of contents

Before we start 1

Exercise 1: types, and how R converts between them 2

Exercise 2: the structures 3

Exercise 3: a text file is just bytes 4

Exercise 4: importing a CSV in two ways 4

Exercise 5: column types 5

Exercise 6: a file that is not comma separated 5

Exercise 7: select what you need and write it back 6

Additional exercises for training 6

Exercise 8: the same data in XML 6

Exercise 9: exploring HTML in the browser 7

Exercise 10: scraping a table from a web page 8

All course material: https://github.com/ASallin/BEcon_3230_Datahandling_ImportCleaningandVisualization

Before we start

Write your code in an R script saved in `2026_Data_Handling/exercises/code`, for example `exercise_03.R`. Open your RStudio project first, so that every path starting with `data/` works. Type the code rather than copying it, and read every error message. If something does not work, raise your hand.

Exercise	What we do
1	Types, and how R converts between them
2	The structures: vector, matrix, list, data frame
3	A text file is just bytes: encoding
4	Importing a CSV in two ways
5	Column types: what the parser guessed, and how to correct it
6	A file that is not comma separated
7	Select what you need and write it back to disk
Training	XML, HTML, and scraping a table from a web page

All the files we use are in the course repository. Either run `git pull` in your copy of it and take them out of `materialsExercises/03_programming_data/data`, or let R download them into your own `data` folder:

```
repo <- paste0("https://raw.githubusercontent.com/ASallin/",
               "BEcon_3230_Datahandling_ImportCleaningandVisualization/main/",
               "materialsExercises/03_programming_data/data/")

files <- c("Air_Quality.csv", "Air_Quality.xml", "temp_anomaly.txt", "hastamanana.txt")

for (f in files) {
  download.file(paste0(repo, f), destfile = file.path("data", f), mode = "wb")
}
```

`mode = "wb"` tells R to write the bytes exactly as they arrive, without changing line endings.

Exercise 1: types, and how R converts between them

In the lecture we saw that every object has a type, and that a vector holds one single type. There are four types you will use constantly:

Type	What it holds	Example
`logical`	`TRUE` and `FALSE`	`TRUE`
`integer`	whole numbers	`3L`
`double`	numbers with decimals	`3.5`
`character`	text	`"St. Gallen"`

If you put values of different types into one vector, R converts them all to the most general type instead of giving an error. This is **coercion**, and it goes in one direction only:

$$\text{logical} \rightarrow \text{integer} \rightarrow \text{double} \rightarrow \text{character}$$

One piece of text in a column of numbers is therefore enough to turn the whole column into text. Predict the answer before you run each line, then run it and compare.

1. What is the type and the class of each of these? `typeof()` tells you how the values are stored, `class()` tells you how R treats the object.

```
typeof(TRUE);  class(TRUE)
typeof(3L);    class(3L)
typeof(3.5);   class(3.5)
typeof("a");   class("a")
typeof(factor("a")); class(factor("a"))
```

Where do `typeof()` and `class()` disagree, and why?

2. R turns logical values into numbers whenever it needs them. What do these return?

```
my_vector <- c(1, 0, 3, -1)
my_vector > 0
as.numeric(my_vector > 0)
sum(my_vector > 0)
mean(my_vector > 0)
```

What do `sum()` and `mean()` of a logical vector tell you?

3. Coercion can happen by itself (implicit) or because you asked for it (explicit). What comes out, and what is the warning telling you?

```
c(1, "2", TRUE)
as.numeric(c("1", "2", "x"))
```

4. Someone stored grades as a factor by mistake:

```
grades <- factor(c("4.5", "5", "6"))
as.numeric(grades)
```

The result is not 4.5, 5, 6. What are those numbers, and how do you get the real values back? Hint: look again at question 1.

Exercise 2: the structures

The type says what the single values are. The structure says how they are arranged. All structures are built out of vectors:

Structure	Dimensions	Types it holds	Use it for
vector	1	one	a single variable
factor	1	categories, with fixed levels	a categorical variable
matrix	2	one	numerical tables, linear algebra
array	3 or more	one	data with more than two dimensions
list	1, elements can be anything	several	a collection of unlike objects
data frame	2	one per column	a data set
tibble	2	one per column	a data frame with nicer behaviour

1. Create the vector `ages <- c(18, 21, 20, 22, 19)`. Ask R how long it is, what type it is, and what the third element is.
2. Make a matrix out of the numbers 1 to 6 with two rows:

```
m <- matrix(1:6, nrow = 2)
m
dim(m)
attributes(m)
```

In which order did R fill the grid? What does `attributes()` tell you about the relation between a matrix and a vector?

3. A vector cannot mix types, a list can. Run both and compare:

```
persons <- c("Anna", "Luca", "Sara")
c(persons, ages)

student_info <- list(names = persons, ages = ages)
student_info$ages
```

What happened to the ages in the first case, and why does the list not have that problem?

4. Build a data frame and look inside it:

```
students <- data.frame(
  name = c("Anna", "Luca", "Sara"),
  age = c(18, 21, 20),
  passed = c(TRUE, FALSE, TRUE)
)
students
str(students)
sapply(students, class)
typeof(students)
```

`typeof()` does not say `data.frame`. What does it say, and what does that tell you about what a data frame is?

Exercise 3: a text file is just bytes

In the lecture on data processing we saw that a text file contains only bytes, and that what those bytes mean depends on the encoding you read them with. With the wrong encoding you get no error, you get wrong characters. `hastamanana.txt` contains one short line of Spanish.

1. Read it and print it. What do you see where the "ñ" should be?

```
hasta <- readLines("data/hastamanana.txt")
hasta
```

2. Look at the raw bytes. Which one is the problem?

```
readBin("data/hastamanana.txt", what = "raw", n = 20)
```

Compare them with an ASCII table. The bytes for `H`, `a`, `s`, `t` are ordinary ASCII. One byte is not.

3. The byte is `f1`. In ISO-8859-1, also called Latin-1, `f1` is "ñ". In UTF-8 the same character needs two bytes, `c3 b1`, so a single `f1` is not valid UTF-8. Read the file again, this time saying which encoding to use:

```
readLines("data/hastamanana.txt", encoding = "latin1")
```

4. In one sentence: why is an encoding problem worse than an error message?

Exercise 4: importing a CSV in two ways

`Air_Quality.csv` contains air-quality measurements for New York City, published by the city's Department of Health in the US data.gov catalog: https://catalog.data.gov/dataset/air-quality. Open that page once to see where the file comes from and in which formats it is offered.

1. Import the file twice, once with base R and once with `readr`:

```
aq_base <- read.csv("data/Air_Quality.csv")

library(readr)
aq <- read_csv("data/Air_Quality.csv")
```

2. Inspect what you imported with `head(aq)`, `tail(aq)`, `dim(aq)`, and `glimpse(aq)` from `dplyr`. How many rows and columns are there, and what does one row represent?
3. Compare `names(aq_base)` with `names(aq)`. The file has a column called `Unique ID`, with a space in the name. What did each function do with that name, and what does that mean for the way you refer to the column afterwards?
4. Compare `class(aq_base)` with `class(aq)`. What is the difference between the two objects?
5. `read_csv()` printed a message about column specifications, `read.csv()` printed nothing. Which behaviour do you prefer when you open a file for the first time, and why?

Exercise 5: column types

`read_csv()` reads the first 1000 rows, guesses a type for every column, and reports what it guessed.

1. Print the full specification with `spec(aq)`. Which columns came out as `col_double()` and which as `col_character()`?
2. Look at `Start_Date`. It contains dates such as `06/01/2013`, but it was not read as a date. Why can the parser not recognise it, and what would the file have to look like for `readr` to guess a date by itself?
3. State the type yourself instead of letting R guess:

```
aq_dates <- read_csv(
  "data/Air_Quality.csv",
  col_types = cols(Start_Date = col_date(format = "%m/%d/%Y"))
)
```

Check `class(aq_dates$Start_Date)`, then run `range(aq_dates$Start_Date)`. Which period does the data cover? Run `range()` on the text version too, and explain the difference.

4. Why is it better to fix a type at import than to convert the column afterwards?

Exercise 6: a file that is not comma separated

`temp_anomaly.txt` contains a monthly series of surface temperature anomalies from NASA's GISTEMP analysis, for the coordinates of St. Gallen. Suppose you are writing a thesis on the effect of climate change on consumption behaviour in Switzerland, and you need this series as a proxy for the salience of climate change.

This exercise is adapted from the tutorial in Chapter 1 of Paul Murrell's book, see the syllabus. Read the chapter for a longer introduction to data stored in text files.

1. Open the file in a text editor before you do anything in R. The first lines are not data. How many lines of metadata are there, and on which line does the first observation start?
2. Read the file:

```
ta <- read_tsv("data/temp_anomaly.txt",
               skip = 9,
               col_names = c("month", "temp_anomaly"))
head(ta)
```

3. What is the purpose of the `skip` argument, and why is it set to 9? And why does `col_names` have to be given here?

4. Why do we use `read_tsv()` and not `read_csv()`? Try it with `read_csv()` and look at the result: how many columns do you get, and what is inside them?
5. How many observations does the series contain, and what is the mean anomaly?

Exercise 7: select what you need and write it back

1. `Name` contains the name of the indicator that was measured. List the distinct values with `unique(aq$Name)`. How many are there?
2. Keep the rows with the amount of fine particulate matter pollution (variable `Data Value`, where `Name == "Fine Particulate Matter (PM2.5)"`) in the winter of 2008-09 (`Time Period == "Winter 2008-09"`) in lower Manhattan (`Geo Place Name == "Lower Manhattan"`). Use `subset()`, and note that you can link several conditions with the `&` operator:

```
pm <- subset(aq,
             Name == "Fine Particulate Matter (PM2.5)" &
             `Time Period` == "Winter 2008-09" &
             `Geo Place Name` == "Lower Manhattan")
pm
```

Note the backticks around the column names that contain a space. How many rows are left, and what is the measured value?

3. Save your result as `airquality_manhattan.csv` in your `data` folder with `write_csv()`. Then open the file in a text editor and look at what was written.
4. Now keep every measurement of fine particulate matter in Lower Manhattan, for all time periods, and save it the same way. If you read `Start_Date` as a date in Exercise 5, sort the series by date first.

Additional exercises for training

> **Note**
>
> These three exercises use material from Lecture 5, this morning: XML, HTML, and scraping a web page. Do them at home. The solutions are published with the rest.

Exercise 8: the same data in XML

`Air_Quality.xml` contains exactly the same measurements as the CSV, in XML.

1. Load the file. The package to use here is `xml2`.

```
library(xml2)
doc <- read_xml("data/Air_Quality.xml")
class(doc)
xml_name(doc)
```

What kind of object did you get? Is it a data frame?

2. Every measurement sits in a `row` element inside the root. Collect them and count them:

```
rows <- xml_find_all(doc, "//row/row")
length(rows)
```

Compare the number with the number of rows in the CSV.

3. Inspect the first measurement: the names of its children, and their contents.

```
xml_name(xml_children(rows[[1]]))
xml_text(xml_children(rows[[1]]))
```

4. Extract one variable for all measurements with an XPath expression, for example every `data_value`. What type is the result, and what do you have to do before you can compute the mean?
5. Compare `start_date` here with `Start_Date` in the CSV, and compare the size of the two files with `file.size()`. What does XML give you that a CSV cannot, and what do you pay for it?

Exercise 9: exploring HTML in the browser

HTML is the code behind most websites, and it is also the data a scraper reads. Here we look at it in the browser before we read it with R.

1. Open a news website that you read, in the browser of your choice.
2. Look at the HTML source code of the page. Depending on the browser this is *View page source*, `Ctrl+U`, or a right-click on an element followed by *Inspect*. Try to recognise the parts of the code that correspond to what you see on the screen.
3. Pick a headline and find it in the source code. `Ctrl+F` searches inside the source. An article usually looks roughly like this: a container, a link, a heading, and an image.

```
<article class="story">
  <a href="https://news.example.com/politics/some-article">
    <img src="https://img.example.com/photo.jpg" alt="A headline">
  </a>
  <h3 class="title">
    <a href="https://news.example.com/politics/some-article">A headline</a>
  </h3>
</article>
```

4. In the *Inspect* view, replace the headline you picked with a title of your own. In most browsers you double-click the text, type, and press Enter. What happens? Did the website change? Did it change for anybody else?
5. Can you replace the article image as well? This is easiest with the address of an image that is already online, which you paste into the `src` attribute.
6. What does this tell you about the difference between what a website shows and what the data behind it is?

Exercise 10: scraping a table from a web page

In the lecture we saw that an HTML document is a semi-structured data source. This exercise has the basic steps for importing data from an HTML table into R.

The aim is a CSV file with data on divided government in US politics, from this Wikipedia page: https://en.wikipedia.org/wiki/Divided_government_in_the_United_States. The page has a table with the president's party and the majority party in the House and in the Senate per Congress, that is per two-year period. The first rows of the cleaned data should look like this:

```
# A tibble: 4 x 4
   year president senate house
  <dbl> <chr>     <chr>  <chr>
1  1861 Lincoln   R      R
2  1862 Lincoln   R      R
3  1863 Lincoln   R      R
4  1864 Lincoln   R      R
```

Start by fixing the paths and loading the packages:

```
library(rvest)
library(dplyr)
library(stringr)
library(readr)

SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"
```

1. Fetch the whole HTML document from Wikipedia with `read_html()`.
2. Extract the tables with `html_table(fill = TRUE)`. The result is a list, so look at the candidates and find the one with `Year`, `President`, `Senate` and `House`. Store it as `tab`.
3. Check the last row. If it contains a footnote instead of data, remove it.
4. The column `Year` contains ranges such as `1861–1863`, but we want one row per year. Write a loop that turns each range into one row per year. `strsplit()` cuts a string into pieces, `unlist()` flattens the result, and `bind_rows()` stacks the pieces together. See `?strsplit` and this introduction to regular expressions for the background.
5. Remove Wikipedia's footnote markers, `[a]`, `[b]` and so on, with `str_remove_all(x, "\\[[^\\]]*\\]")`.
6. Inspect the result and write it to a CSV file, using `OUTPUT_PATH`.